시스템 장애 분석 및 트러블슈팅 플레이북

SRE 관제 상황실 기록부: Memory, CPU, Deadlock

[WSL2 Ubuntu] [Agent-App] [Troubleshooting Taskforce]

Gemini Notebook

선배님! 실서버에서 프로세스가 죽으면 당장 고객 항의가 쏟아지니까 일단 'systemctl restart'부터 누르고 싶어지잖아요?

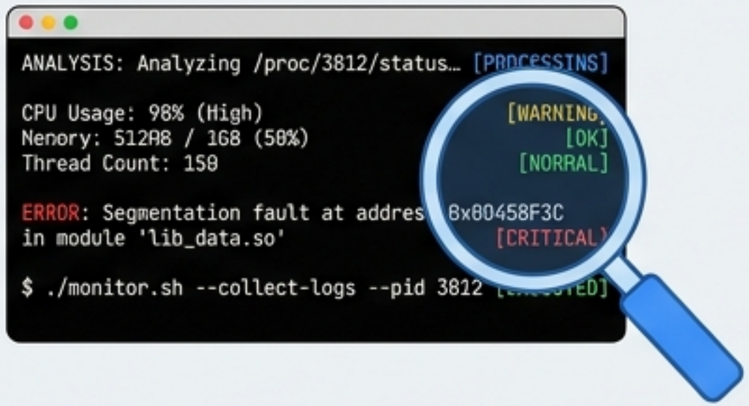
```
$ systemctl restart process-service  [FAILED]
```



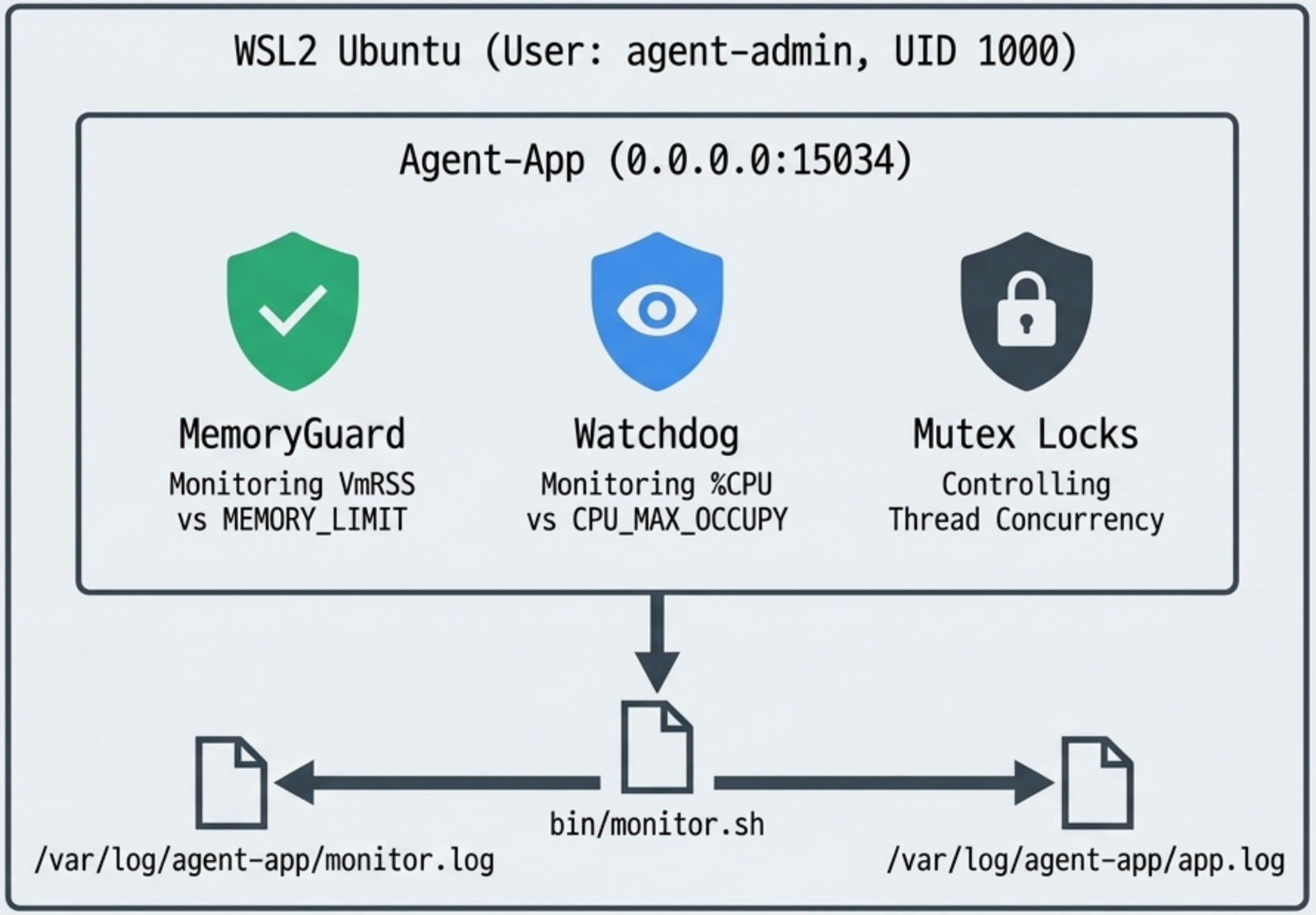
그 마음 백번 이해하지. 하지만 로그와 관제 데이터를 확보하지 않고 재부팅하면, 불이 났는데 발화 지점을 안 보고 물만 뿌린 것과 같아. 같은 장애가 내일 또 터지지.

이번 B4-02 과제에서 우리가 관제 스크립트('monitor.sh')와 '/proc' 파일시스템을 먼저 뜯어본 이유가 바로 그거야.

```
LOG: process-service crash detected.    [INFO]
Timestamp: 2024-01-26 10:45:32 UTC
```



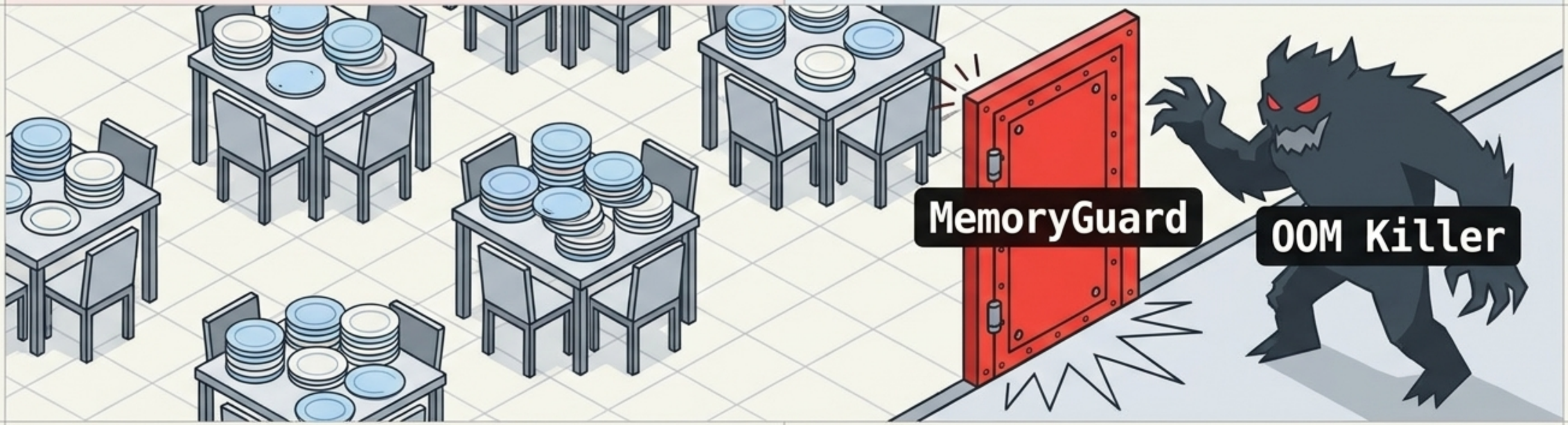
Gemini Notebook



Gemini Notebook

메모리 누수(Memory Leak)는 왜 개발자들이 ‘침묵의 살인마’라고 부르나요?

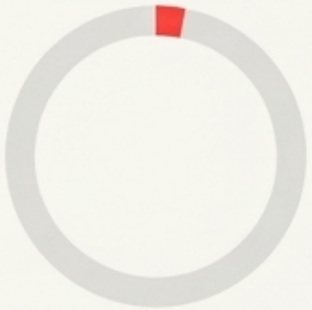
처음 1~2초는 멀쩡해 보이거든. 하지만 빈 그릇을 안 치우는 식당처럼 손님 자리가 없어지지.
리눅스 커널은 메모리가 고갈되면 OOM Killer가 튀어나와 무차별적으로 데몬을 사살해.
그래서 ‘MemoryGuard’가 사살당하기 전에 자결(‘exit 137’)한 거야.



Gemini Notebook

🚨 Case 1: OOM Crash 실측 데이터 비교

MEMORY_LIMIT=60 MB



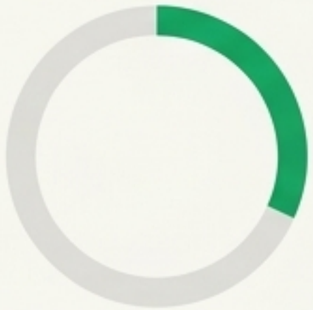
생존 시간: 6.80초 (Fast Crash)



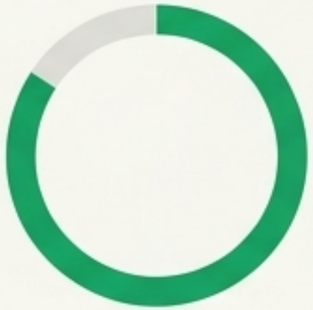
피크 메모리: 61 MB

```
[MemoryGuard] SELF-TERMINATED ... exit 137
```

MEMORY_LIMIT=200 MB



생존 시간: 25.19초 (270% ⬆)



피크 메모리: 202 MB

```
[MemoryGuard] SELF-TERMINATED ... exit 137
```

Gemini Notebook

와! 'MEMORY_LIMIT'을 60MB에서 200MB로 늘렸더니 생존 시간이 270% 늘었어요! 해결된 건가요?

아니지! 그건 침몰하는 배의 물받이 양동이 크기만 키운 거야. 언젠간 또 가라앉아. 전역 버퍼 누수 코드를 지우는 게 진짜 근본 해결이지.

임시 조치 (Workaround)



환경변수 'MEMORY_LIMIT' 상향

근본 해결 (Root Fix)

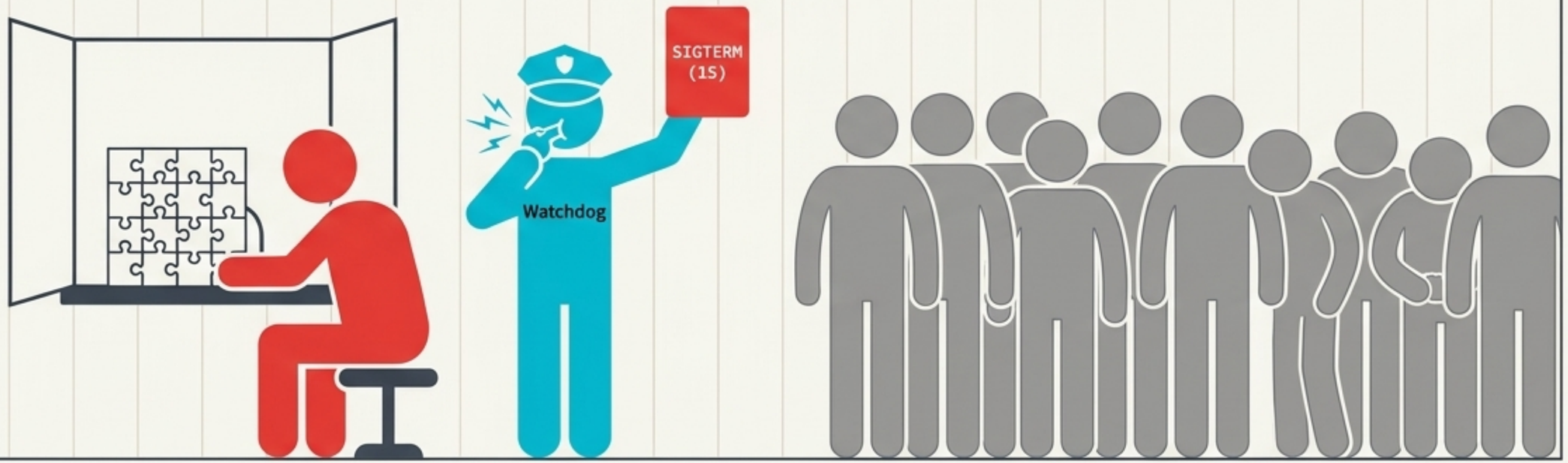


전역 버퍼 누수 코드 제거 및 객체 파기

Gemini Notebook

CPU 과점유 테스트 때 ‘WATCHDOG: Action: Sending SIGTERM’ 로그가 찍히면서 프로세스가 종료되었어요. 에러인가요?

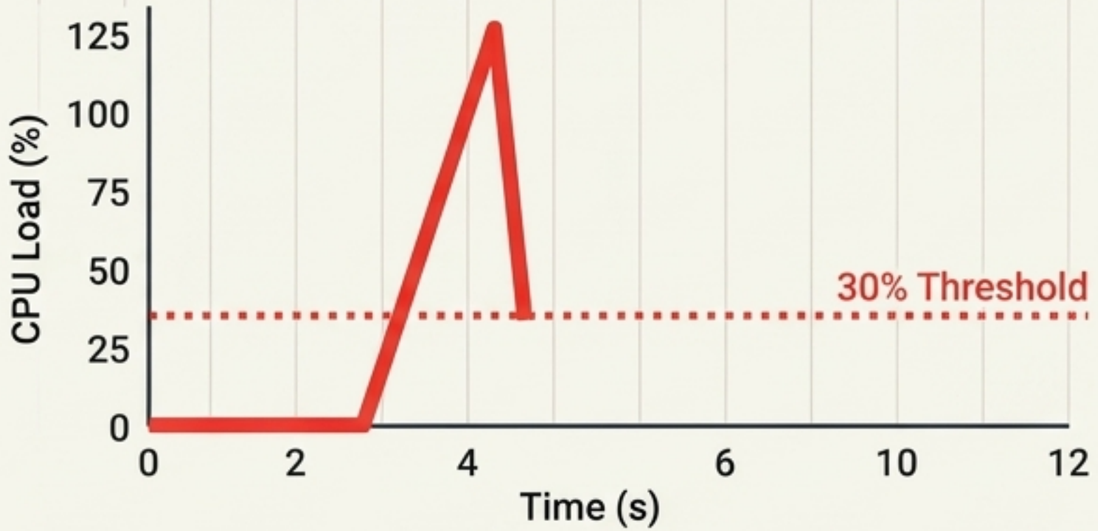
아주 훌륭한 시스템 보호 조치야.
은행 창구를 1명이 독점하면 뒤 손님들이 얼어붙잖아.
Watchdog이 ‘다른 프로세스들 숨 좀 쉬게 일단 퇴근해!’ 하고 ‘SIGTERM’ 신호를 날려준 거지.



Gemini Notebook

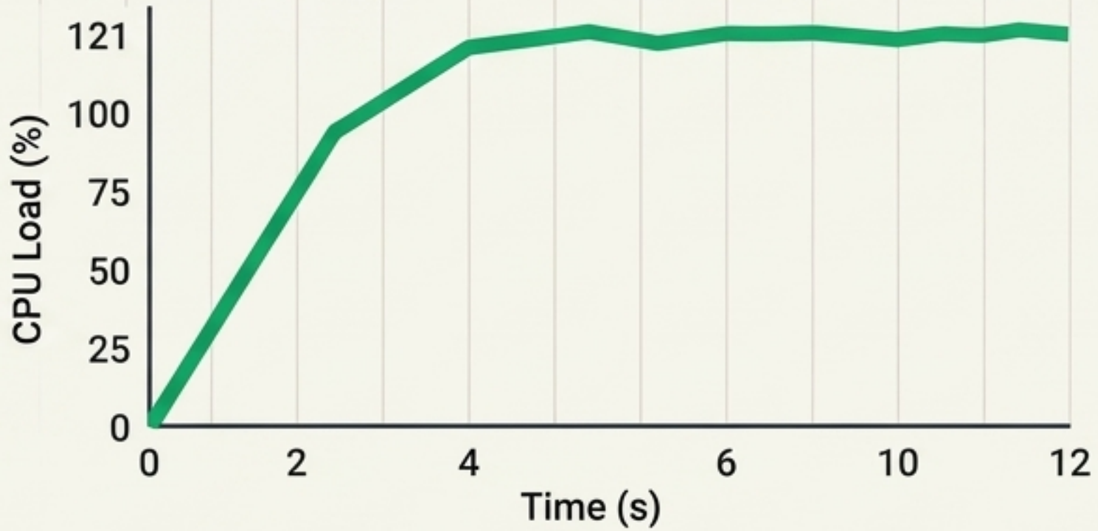
⚡ Case 2: CPU Spike 실측 데이터 비교

CPU_MAX_OCCUPY=30%



생존 시간	2.21초 후 강제 종료
피크 CPU	122%
Watchdog 상태	SIGTERM Sent

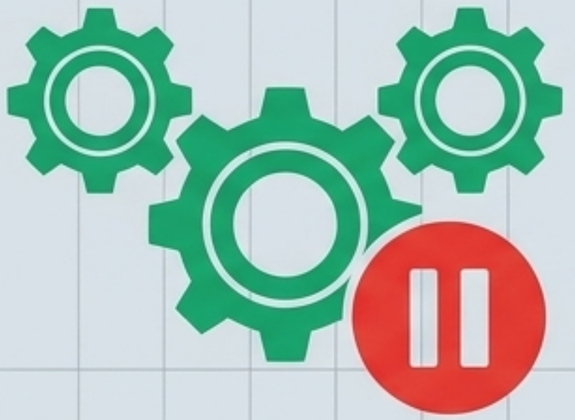
CPU_MAX_OCCUPY=95%



생존 시간	> 4.60초 (영구 지속)
피크 CPU	121%
Watchdog 상태	No Signal (Safe)

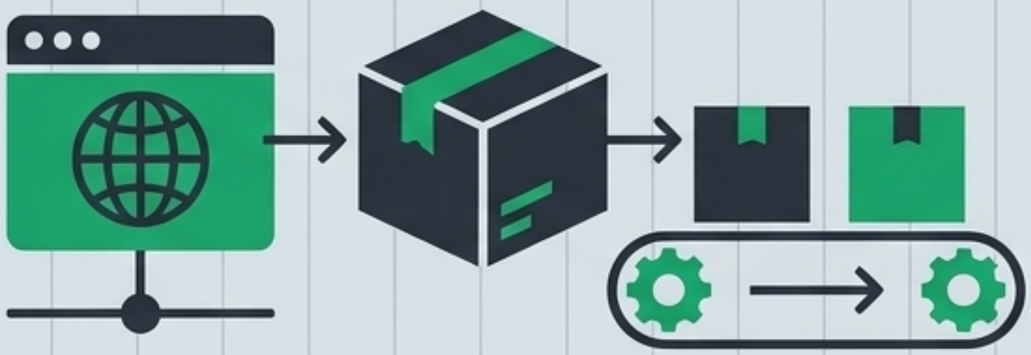
Gemini Notebook

한도 완화(Workaround)를 넘어선 진짜 최적화



연산 루프 최적화

타이트 루프 내에 'time.sleep(0.01)' 삽입.
OS 스케줄러에게 CPU 타임 슬라이스를 양보(Yield)하여
Context Switching 허용.



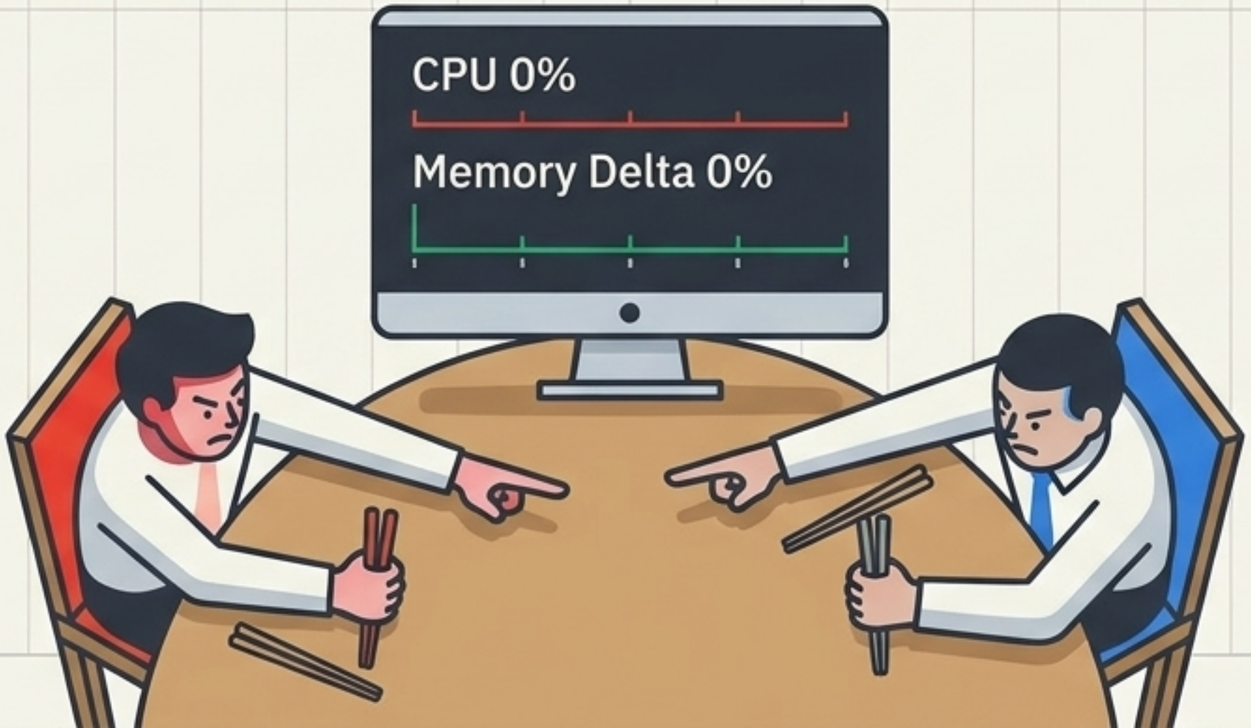
Worker Offloading 아키텍처

메인 프로세스에서 연산 분리.
Celery, Redis Queue 등 비동기
백그라운드 태스크 워커로 부하를 위임.

Gemini Notebook

선배님, 데드락은 왜 CPU도 0%고
로그도 안 찍히는데 PID는 살아있나요?

이게 제일 무서운 좀비 상태야. 두 스레드가
서로 쥐고 있는 젓가락을 내놓으라고 째려보며
무한히 잠들어(Futex Wait) 있는 거지.



Gemini Notebook

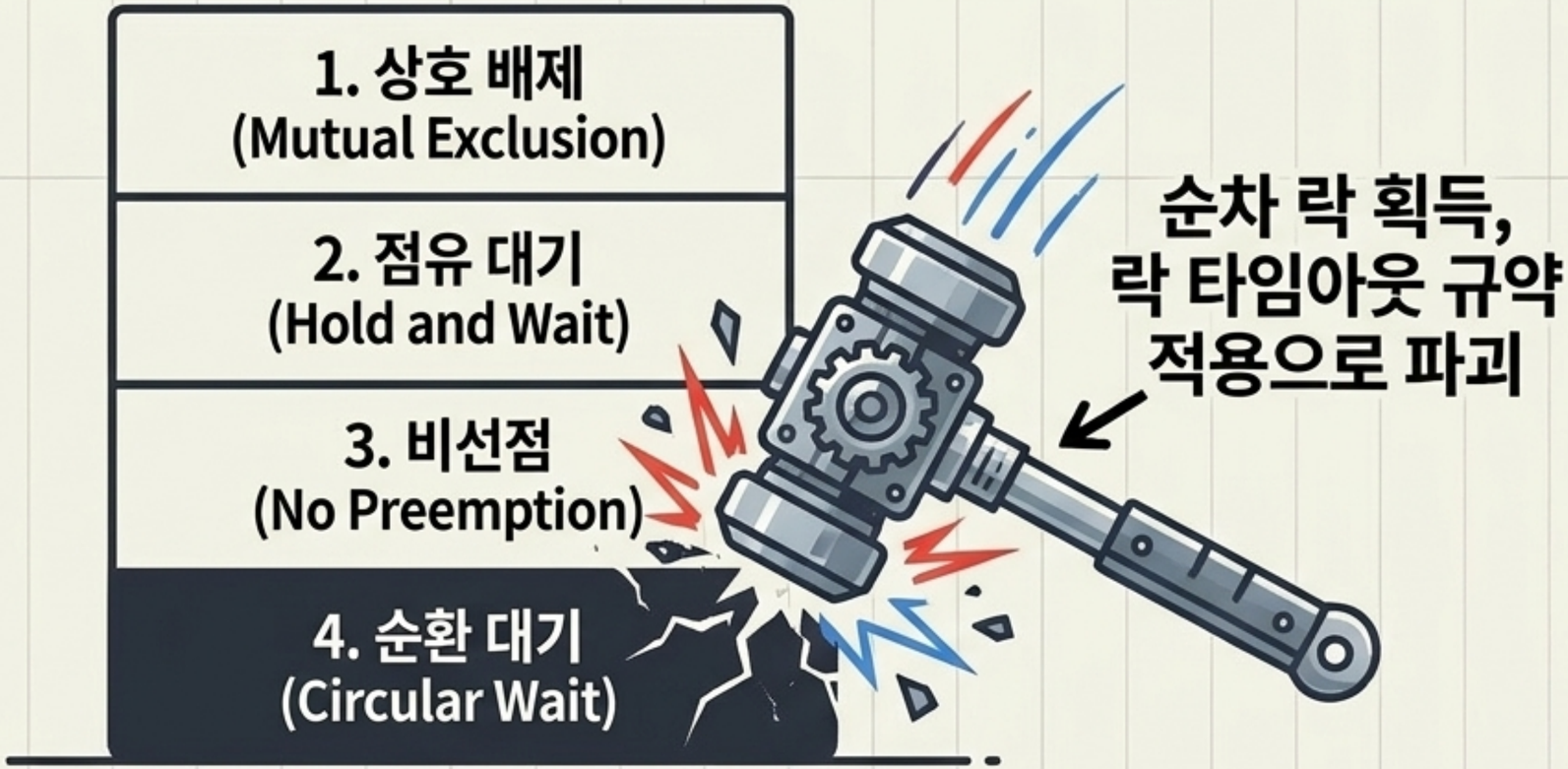
🔒 Case 3: Deadlock 진단 실측 데이터 비교

MULTI_THREAD_ENABLE=true		MULTI_THREAD_ENABLE=false	
			
상태	Circular Wait LOCKED	상태	Sequential Avoided
리소스	0% CPU, 0% MEM Delta (Alive / Frozen)	리소스	정상 활동 (Alive / Responsive)
마지막 로그: [Thread-WorkerA] WAITING for lock_beta... BLOCKED		마지막 로그: [Safe-Worker] All locks processed sequentially	

Gemini Notebook

코프먼 4대 조건 깨뜨리기 (Root Fix)

데드락은 4대 조건 중 하나만 깨뜨려도 사라진다. ‘MULTI_THREAD=false’로 바꾸자마자 한 번에 한 명씩 순서대로 락을 쥐니까 ‘순환 대기’가 깨지면서 말끔히 풀렸잖아!



Gemini Notebook

🎯 3대 시스템 장애 통합 진단 매트릭스 (SRE Cheat Sheet)

장애 유형	관제 증거 (Evidence)	시스템 보호 조치	임시 조치 (Workaround)	근본 해결 (Root Fix)
OOM Crash (Memory)	RSS 선형 증가	MemoryGuard ‘exit 137’	‘MEMORY_LIMIT’ 상향	누수 객체 제거 및 캐시 도입
CPU Spike (Latency)	‘top’ 점유율 급증	Watchdog ‘SIGTERM 15’	‘CPU_MAX_OCCUPY’ 상향	‘sleep()’ 양보 및 백그라운드 워커 분산
Deadlock (Freeze)	CPU 0%, PID 생존, Futex Wait	(없음, 영구 정체)	단일 스레드 전환 (‘MULTI_THREAD=false’)	락 획득 순서 강제, 타임아웃 적용

Gemini Notebook

🎯 3개월 차 주니어가 반드시 깨달아야 할 핵심 원칙 3가지

1

재부팅 버튼보다 터미널 관제 로그 수집이 먼저다

‘ps -ef’, ‘top -H’, ‘monitor.log’ 등 증거를 확보해야 재현 경로와 근본 원인을 증명할 수 있다.

2

임시 조치(Workaround)와 근본 해결(Root Fix)을 명확히 구분하라

환경변수 완화는 서비스를 살려두기 위한 응급처치일 뿐. 객체 누수 제거와 락 순서 강제가 뒤따라야 한다.

3

데드락은 코프먼 4대 조건 중 하나만 깨뜨려도 사라진다

순환 대기나 점유 대기를 깨뜨리는 순차 락 획득 또는 타임아웃 규약이 동시성 프로그래밍의 핵심이다.

Gemini Notebook

재부팅 버튼을 누르기 전, 관제 데이터가 가리키는 진실을 먼저 읽어내십시오.

시스템 장애는 이유 없이 발생하지 않으며,
로그는 항상 흔적을 남깁니다.

Gemini Notebook